澄灯 LumaRV 面向控制板试产与返修的语境感知双核调试系统

摘要

控制板完成固定测试后，工程师还会临时检查某一路输出，并要求保留其他状态。澄灯 LumaRV 在开发板上实现这类中文请求的处理：主核完成语境与工具判断、参数检查和寄存器读回，副核在私有存储中运行 CoreMark。针对片上资源，我们设计位置语义分支、受约束工具残差和 4/8MAC 两类计算单元。指定中文 PWM 样本的完整模型计算由 658.079 ms 降至 54.640 ms，整数输出一致。系统在 PG2L200H 上以 54 MHz 运行，主核 CoreMark/MHz 中位数 4.038437，双独立实例累计吞吐 7.780660/MHz。本文给出模型训练、资源与时序以及实板指令结果。

第一部分 作品概述

1.1 功能与特性

系统支持结温与周期计数读取、LED 整体及单灯位操作、1kHz PWM 占空比设置和 CoreMark。LED/PWM 写入后读取寄存器，核对请求值与保存值。多动作、参数越界及未开放能力的请求拒绝执行。副核可后台运行计算检查，主核继续处理交互。

1.2 应用领域

一块控制板完成固定测试后，工程师根据故障疑点追加复测：先设置一个已知输出，再单独改变其中一位，要求其他通道保持；或微调 PWM 占空比，核对请求值与寄存器读回。固定流程仍用脚本，中文入口处理临时请求及其中

的限制。我们用开发板自身验证这条操作链路，副核后台运行 CoreMark 时，主核完成中文 PWM 请求与回读。

频率超限可作为这条链路的扩展场景：在信号源或驱动板联调时，外部测量模块上传输出频率，上位机按工艺阈值提示超限；工程师确认后下发限频请求，板端检查工具与参数，执行后读回寄存器，再由外部测量模块复测输出。这样将发现问题、调整配置和复测结果记在同一条记录中。当前原型已验证中文调参、能力检查和写后读回；PWM 固定 1kHz，测频与动态限频仍需接入相应模块。

1.3 主要技术特点

系统由两颗五级 RV32IM 软核、各自 Cache 和存储、4MAC 字节编码器 8MAC 位置语义侧核及外设组成。主核运行交互、整数模型和执行接口；副核使用 64 KiB 私有 RAM，不访问 AI 引擎和外设。模型结合学习型短语记忆、位置语义分支与小型工具残差。

1.4 主要性能指标

表 1 主要性能指标

指标	结果
主频 / CPU 性能	54 MHz 双核；主核约 4.1 双核累计约 7.78 CoreMark/MHz
指定中文 PWM 模型	54.640 ms；同镜像软件加速 12.04 倍
轻量路径往返中位数	1.32～3.66 ms；四句各 n=10
学习权重 / 存储	197,816 B；主存 512 KiB、W1 缓存 128 KiB、副核 64 KiB
布局布线资源	55,277 LUT、39,035 FF、342.5 DRM、40 APM

1.5 主要创新点

（1）位置感知语义模型。一句话提到外设，可能是在下命令，也可能只是解释或否定。我们在字符短语记忆之外，加入全局与前后半句的位置特征，经

2048→64→16 分支判断语境和工具。语境与工具分支使用 25,014 条训练文本，覆盖合法操作、否定、转述、多动作和能力边界。

（2）受约束工具残差。将位置分支的 64 维隐层与编码器六类分数组成 70 维输入，生成五类合法工具的修正量。当前候选不是拒绝、前两类分差不超过 256 时，才加回修正；后续语境否决与参数许可仍然有效。它用于减少合法工具混淆，不承担开放式回答或多工具规划。

（3）异构低比特推理。字节编码器使用 4MAC，位置语义头使用 8MAC；语义头第一层的 128 KiB INT8 权重常驻八个 bank，并跳过全零特征组。指定中文 PWM 样本的完整模型由同镜像软件 658.079 ms 降至 54.640 ms，输出数组一致。双核分工使后台 CoreMark 与前台 AI、IO 操作可在已测任务中并行。

1.6 设计流程

从五类板卡操作与参数范围建立语料，训练并导出整数参数；实现 CPU、Cache 和两类乘加单元，对照 C 参考验证数值；经 PDS 布局布线与时序分析后，在开发板运行中文请求、写后读回和双核基准。

第二部分 系统组成及功能说明

2.1 整体介绍

图 1 给出一次追加复测。主核逐条处理请求，副核在独立存储中运行 CoreMark。已有实板记录覆盖副核忙碌期间的完整中文 PWM 模型与寄存器读回，模型数组与单独运行一致。

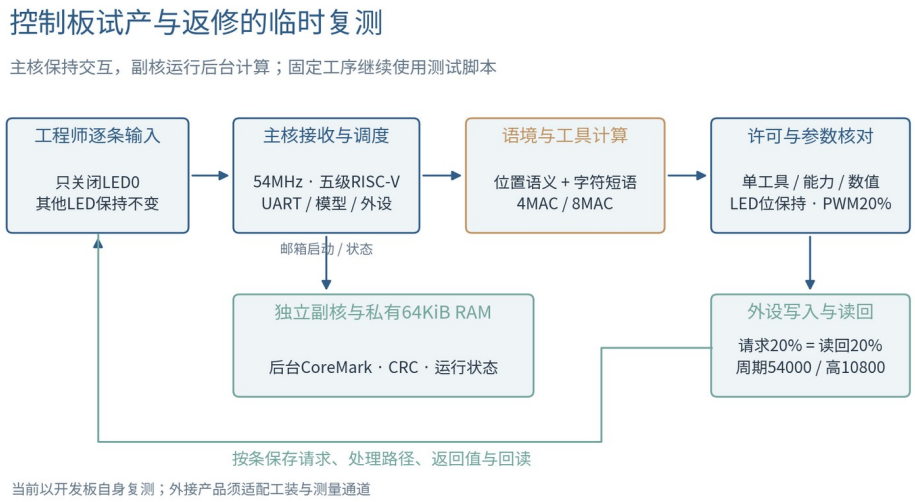


图 1 控制板临时复测的交互与读回流程

固定测试保留脚本。临时请求逐条执行，结果包含处理路径、参数与读回；要求改变未开放的 PWM 频率时拒绝。产品合格判定仍需相应工装和外部测量。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

PG2L200H 以 27 MHz 晶振经 GPLL 生成 54 MHz 系统时钟。主核连接 UART、片内温度、GPIO、PWM 与 32 位周期计数，主 SRAM512 KiB、语义 W1 缓存 128 KiB。副核配 64 KiB RAM、32 KiB 指令 Cache 及 4 KiB 数据 Cache，经邮箱返回状态、周期和 CRC。两核 BTB 分别 512 和 64 项，模式表与返回地址栈另计。

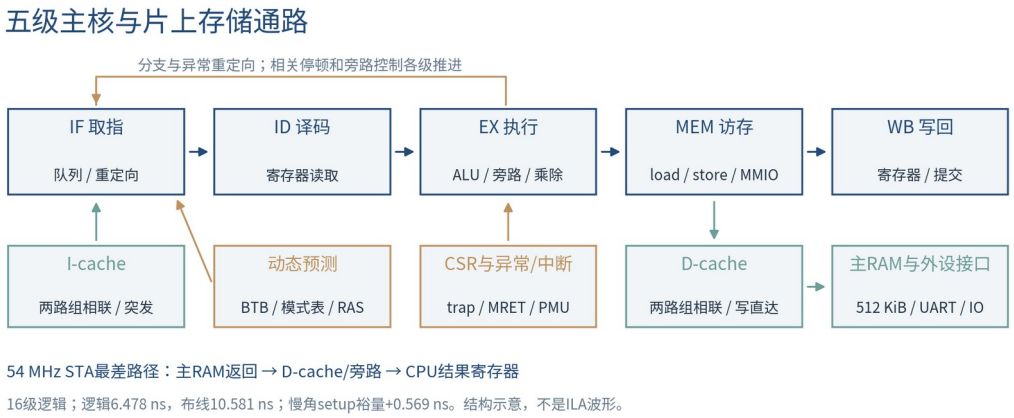


图 2 主核五级流水线与片上存储通路

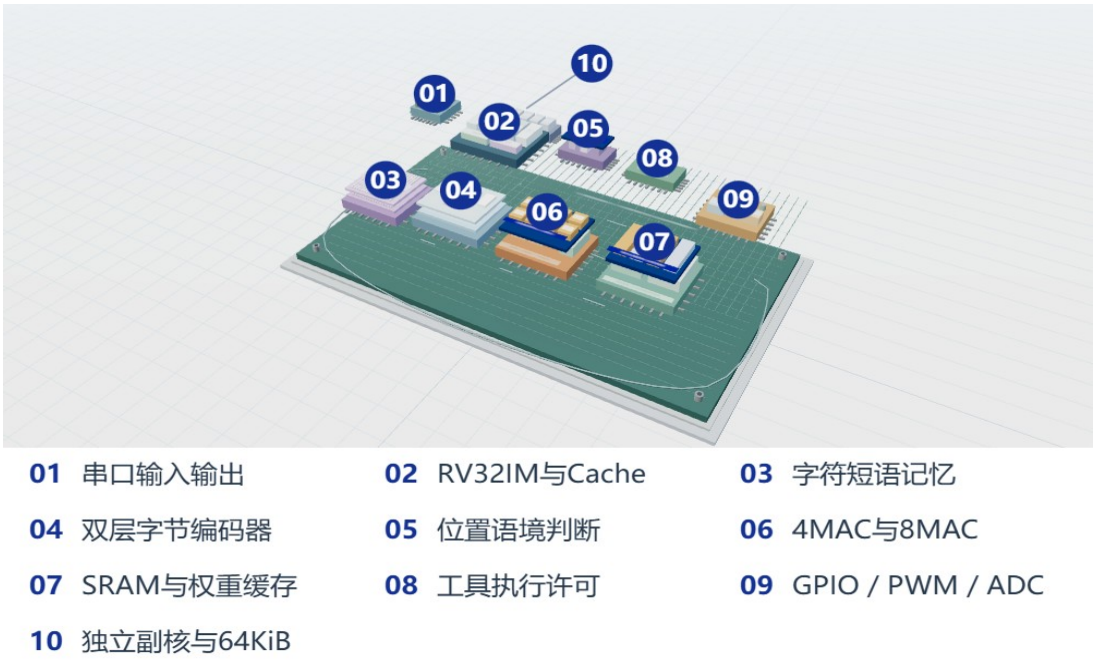


图 3 模块分层与片上计算结构

三维图按处理器、语义计算、存储和外设分层，用于说明逻辑连接与请求路径。网页可旋转、展开模块，PPT 保留可交互模型；三维动画不参与性能计时。

2.2.2 机械设计介绍

使用厂家开发板、配套电源、USB 串口和 JTAG 线缆。展示机位使板卡 LED 与电脑返回结果可以同框观察。

2.2.3 电路各模块介绍

表 2 硬件模块与接口

模块	关键输入	关键输出
CPU 与 Cache	时钟、复位、访存响应	指令/数据地址、写数据、提交 PC
共享 DMA	计算任务、地址与读写请求	数据响应、任务归属、错误状态
4MAC 编码器	INT4 权重、INT8 激活	INT32 累加值、任务状态
8MAC 位置语义头	INT8 特征、常驻权重	INT32 语义分数、完成状态
UART 与外设	串口输入、按键、ADC 采样	串口输出、LED、PWM 寄存器值

PWM 固定 1kHz，周期与高电平计数共同决定占空比。LED 整体掩码与单位置位/清位均返回读回值。

2.3 软件系统介绍

2.3.1 软件整体介绍

控制台提供完整 AI 请求、快速控制、后台计算检查和 CPU 基准入口。模式切换收到确认后才发送工具请求；每条结果与就绪状态对应。双核基准分别显示各核成绩，并按共同时间计算累计吞吐。

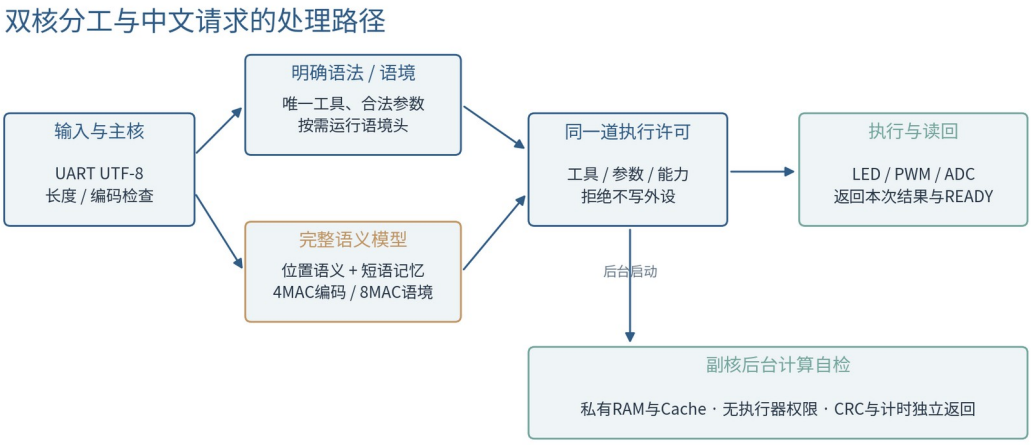


图 4 中文请求的处理流程

默认混合模式先对严格机器语法及部分明确中文命令做全句匹配和参数检查，命中后直接解析，不运行学习语境头；其余中文先运行语境头，再按需调

用完整编码器。执行接口最终校验请求状态、参数和工具可用性，并返回读回结果。页面标记本次实际运行的路径。

2.3.2 模型设计与训练

模型任务是判断执行语境和工具类别，不生成自由文本。工具包括温度、PWM、CoreMark、计数器和 LED，另有拒绝类别；数值参数由固件解析。低比特工具调用与字符短语记忆参考 Needle，具体来源与本项目实现见下表。

表 3 参考基础与本项目设计

参考基础	本项目设计
Needle 低比特工具调用与字符短语记忆	板卡五类操作、中文语境分支与训练语料
通用 Transformer 编码器结构	双层 d64 整数运行时、首尾位置特征与 70→5 工具残差
RISC-V 指令集	五级 CPU、Cache、异常中断与外设实现
整数乘加与同步 RAM 方法	4MAC 编码器、8MAC 位置语义侧核、常驻权重缓存与零组跳过
MASSIVE 中文语料	选择 2,800 条域外请求，按设备能力重新标注

字节编码器为两层 d64、四注意力头，含学习型 2/3-gram 记忆。位置分支按 Unicode 字符中点划分前后两段：全局 1024 维，前后各 512 维，输入 2048→64→16 语义头。64 维隐层与六类编码器得分经缩放和截断后组成 70 维输入，由 CPU 计算 70→5 残差。门控控制修正是否加回，语境否决仍然优先。

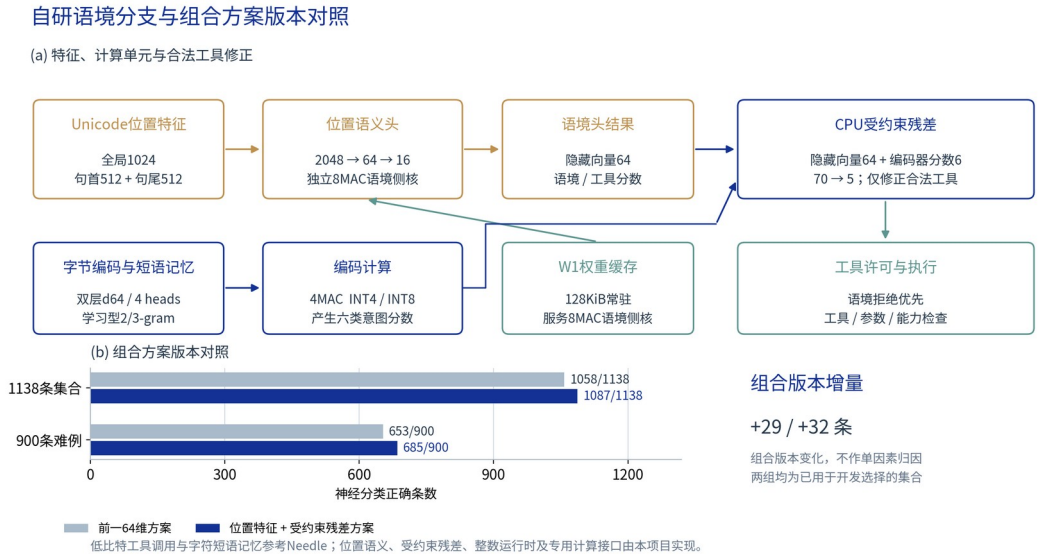


图 5 自研语义分支与组合版本对照

组合版本在 1138 条开发集合中由 1058 条正确增至 1087 条，在 900 条难例中由 653 条增至 685 条。结构、训练及选择过程同时变化，这组版本对照不能分配为位置分支或残差的单独收益。

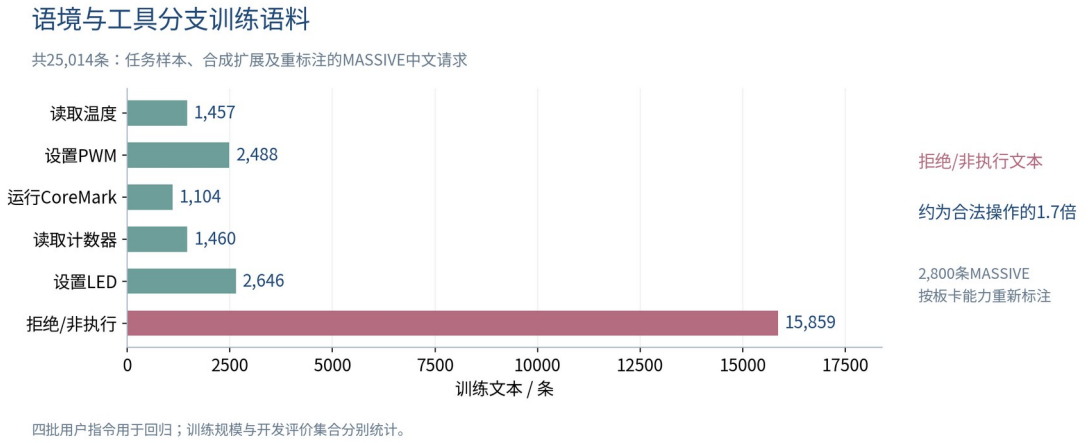


图 6 语境与工具分支的训练文本分布

语境与工具分支使用 25,014 条训练文本：合法操作 9,155 条，拒绝及非执行表达 15,859 条。语料包含任务样本、合成语境扩展和 2,800 条按板卡能力重标注的 MASSIVE 中文域外请求。五类合法操作分布见图 6。四批用户指令用于回归，训练规模并非仅有 25 句。

197,816 B 学习权重导出为 C 数组并编入主核固件。4MAC 编码器采用 INT4/INT8 权重，8MAC 位置语义头采用 INT8 权重，两者 INT32 累加。语义头

W1 为 2048×64 字节，分八个 bank 常驻 128 KiB 缓存；首次经 32 位总线装载，计算时各 bank 每拍提供一个字节。

表 4 软件模块与输入输出

软件模块	输入	输出
输入处理	UTF-8 文本、长度及帧校验	单条请求或输入错误
语境与工具模型	字符、位置特征及编码向量	语境判断、六类分数
参数与许可	原文、工具候选、硬件能力	工具、参数及操作类型
执行与显示	允许执行的请求	实际值、读回结果、计时及就绪

第三部分 完成情况及性能参数

3.1 整体介绍

实板已完成中文请求、LED/PWM 读回和双核基准。副核后台忙碌期间主核完成完整 PWM 模型，重复启动被拒绝；十种初始 LED 掩码下的 20 项位保持检查通过。图 7 为开发板实物，图 8 展示供电与 JTAG 连接。

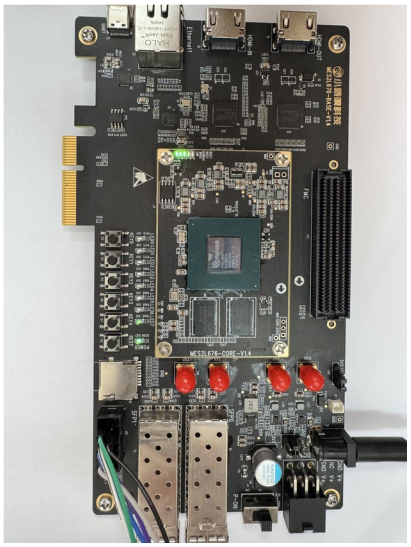


图 7 开发板实物全貌



图 8 供电与 JTAG 连接

3.2 工程成果

3.2.1 机械成果

采用开发板现有结构与连接件，展示布局以看清板卡、线缆和用户 LED 为准。

3.2.2 电路成果

器件汇总使用 55,277 LUT、39,035 FF、342.5 个 DRM36 等效单元、40 个 APM、1 个 ADC 和 1 个 GPLL。下表按原始层次报告拆分 CPU、AI 及副核，父子模块不重复相加。

、

层次范围	LUT	FF
主核 CPU 及 I/D Cache	24,330	17,581
4MAC 及 8MAC 计算区	8,678	6,987
副核子系统	18,704	13,866
外围、存储接口与顶层余项 [1]	934	601
层次表合计	52,646	39,035

表 5 层次资源使用

余项由层次总数扣除前三项得到。原始层次 LUT 合计 52,646，与器件汇总 55,277 口径不同；CoreMark/LUT 使用器件汇总。

3.2.3 软件成果

PWM 请求 20%时，读回 20%，周期 54,000、高电平 10,800、使能 1。副核状态单列，完成与 CRC 返回后才显示基准结果。图 9 摘录既有实板返回。

中文PWM请求的实板返回

输入	请将PWM占空比设置为20%。
计算配置	54 MHz · config6 · 完整模型
模型计算	2,950,565周期 / 54.640 ms
执行与寄存器读回	请求20% / 读回20% / match=1
PWM计数	周期54,000 / 高电平10,800 / 使能1
并行检查	副核忙碌期间主核完成AI；模型数组与单独运行一致

图 9 中文 PWM 请求的计算与寄存器读回结果

3.3 特性成果

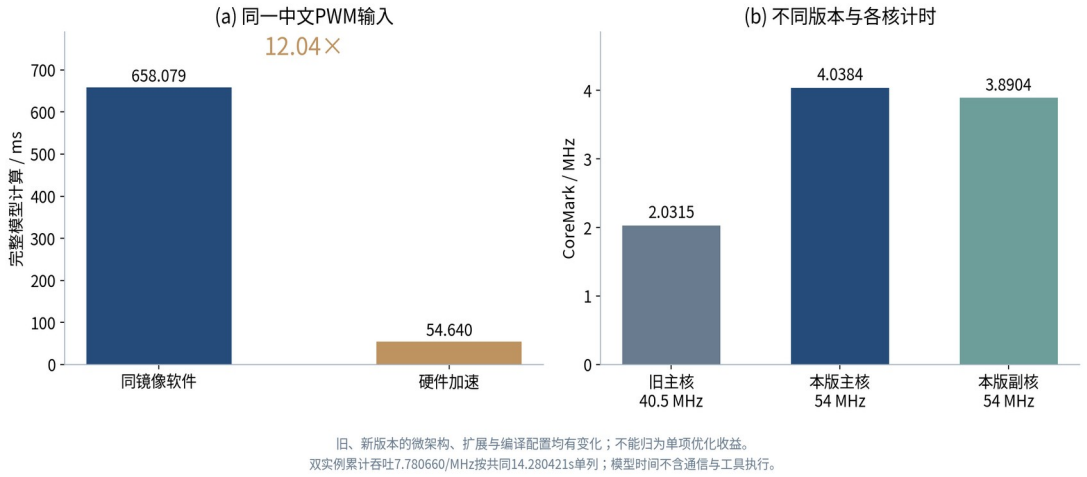


图 10 指定中文 PWM 请求的完整模型软件与加速路径

同一镜像、同一中文 PWM 输入的完整模型：软件 35,536,262 周期、658.079 ms，加速路径 2,950,565 周期、54.640 ms，加速 12.04388 倍。编码器、语境、工具和残差数组一致；一次冷缓存记录 56.718 ms。

表 6 应用计时范围与实测结果

计时对象	实测结果	范围
指定中文 PWM 完整模型	54.640 ms	板上计算，不含串口与工具执行
机器语法 / 语境分支	1.32～3.66 ms	四条固定输入各 10 次原生 UART 往返中位数
双核计算自检	共同时间 14.280421s	每核 3000 迭代；两核 CRC 均通过

主核 19 次基准每次 3000 迭代，均通过 CRC，耗时 13.756066～13.762035 秒，CoreMark/MHz 中位数 4.038437。副核 3.890411/MHz。双实例从同一次邮箱启动到较晚的结束标记，共同 14.280421 秒；6000 次迭代对应 7.780660/MHz、420.155666 次/秒，按器件汇总资源为 0.00760091 CoreMark/LUT。共同窗口包含启动与结束标记开销。

两组开发集合分别包含 196 条合法/942 条拒绝请求，以及 200 条合法/700 条拒绝请求。集合已用于开发选择，表中统计神经分支的分类结果。

表 7 原始神经分类结果

评价集	神经分类正确	误接受	误拒绝	合法工具错分
1138 条	1087/1138（95.52%）	50	0	1
900 条难例	685/900（76.11%）	190	24	1

1138 条集合中的 50 次误接受和 900 条难例中的 190 次误接受，说明神经分支仍会把非执行请求判成可执行。合法工具错分单列，不能与误拒绝混为一项。

混合路径分别 1135/1138、884/900 正确，合法请求分别 193/196、184/200；这两组开发集合中的应拒绝请求均未放行。混合结果包含语境、明确语法和参数/能力检查，不计作神经分类准确率。

四批用户原文实板结果依次 24/25、25/25、25/25、25/25。一条 81 字节输入按协议上限拒绝；64 字节以内 99 条中 98 条符合预期，一条 CoreMark 成绩询问被保守拒绝。强制完整模型另有一条 LED0 置位误拒绝：编码器选对 LED，语境拒绝边际 7 超过阈值 5；默认混合路径通过明确 LED 语法执行，未运行模型。拒绝前后 LED/PWM 配置保持。

主核 PMU 样本 743,124,772 周期、退役 712,638,923 条，CPI 约 1.043。CoreMark 以独立算法计时，各核分别运行私有存储中的实例。全部基准算法使用同一编译配置，源码保持原样；应用与 AI 另用 GCC14 O2。微架构、标准扩展与编译策略共同变化，现有结果不作单因素归因。

表 8 编译工具链与选项

编译对象	工具链	主要选项	指令范围
应用与 AI	GCC14	-O2	RV32IM、CSR、FENCE.I
两核 CoreMark 全部算法	GCC15	-O3；-funroll-all-loops；-mtune=rocket	上述基础加 Zba/Zbb/Zbc/Zbs/Zicond

编译选项和标准扩展已启用，但未逐项测量扩展收益。构建入口为 05_tools/build_dual_firmware.py，两个实例各 3000 次。

54 MHz 慢/快角 setup 裕量 0.569/7.529 ns，hold 裕量 0.141/0.100 ns。最差慢角路径为主 RAM 读出，经 D-cache 返回和旁路，进入 CPU 结果寄存器；逻辑延迟 6.478 ns、布线 10.581 ns，共 16 级逻辑。

第四部分 总结

4.1 可扩展之处

接入产品须定义型号工具、参数与接口，并配合外部工装测电压、电流和输出波形。应用收益可用同任务操作时间与人工干预次数评价，目前尚未完成这类对比。

4.2 心得体会

最初只把工具类别认出来，仍会遇到解释句、否定句和合法工具混淆。位置特征用于保留文字前后关系，小型残差处理合法类别间的修正；完整模型的 LED0 误拒绝也提醒我们，语境门控会挡住真正想执行的句子。

部署时，权重读法与端口时序同样影响资源。将语义头第一层权重分 bank 常驻，再跳过全零特征组，可以减少重复搬运；计算仍需与整数参考逐项对照。

双核把长计算与交互分开，已有样本证明两项任务可以同时完成。下一步应补独立语言评价、忙闲延迟分布和产品工装验证，而不是只围绕熟悉指令继续提高分数。

第五部分 参考文献

[1] RISC-V International. RISC-V Instruction Set Manual[EB/OL]. https://docs.riscv.org/reference/isa/.

[2] EEMBC. CoreMark benchmark[CP/OL]. https://github.com/eembc/coremark.

[3] Cactus Compute. Needle source repository[CP/OL]. https://github.com/cactus-compute/needle.

[4] Home Assistant. Intents architecture[EB/OL]. https://contributors.home-assistant.io/core/internals/intents/.

[5] FitzGerald J, Hench C, Peris C, et al. MASSIVE: A 1M-Example Multilingual Natural Language Understanding Dataset with 51 Typologically-Diverse Languages[EB/OL]. 2022. https://arxiv.org/abs/2204.08582.

[6] Three.js contributors. Three.js[CP/OL]. https://github.com/mrdoob/three.js.

[7] NI. Electronics Manufacturing Test Solutions[EB/OL]. https://www.ni.com/content/dam/web/pdfs/electronics_manufacturing_test_solutions_brochure.pdf.

[8] Ndubuaku H. We Distilled Gemini Tool Calling into a 26M Model[EB/OL]. 2026-05-12[2026-11-03]. https://cactuscompute.com/blog/needle.

[9] Vaswani A, Shazeer N, Parmar N, et al. Attention Is All You Need[C/OL]//Advances in Neural Information Processing Systems 30. 2017[2026-11-03]. https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[10] Jacob B, Kligys S, Chen B, et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference[C/OL]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2018[2026-11-03]. https://arxiv.org/abs/1712.05877.

[11] RISC-V International. Bit Manipulation Extensions[EB/OL]. The RISC-V Instruction Set Manual, Version 20260120[2026-11-03]. https://docs.riscv.org/reference/isa/v20260120/unpriv/b-st-ext.html.

[12] RISC-V International. Zicond Extension for Integer Conditional Operations, Version 1.0.0[EB/OL]. 2026[2026-11-03]. https://docs.riscv.org/reference/isa/v20260120/unpriv/zicond.html.

[13] Free Software Foundation. Options That Control Optimization: Using the GNU Compiler Collection, GCC 15.2.0[EB/OL].[2026-11-03]. https://gcc.gnu.org/onlinedocs/gcc-15.2.0/gcc/Optimize-Options.html.

[14] Ndubuaku H, Mosoyan K, Mroz J, et al. A Controlled Study of Attention-Only Transformers[EB/OL]. 2026[2026-11-03]. https://arxiv.org/abs/2607.18363.